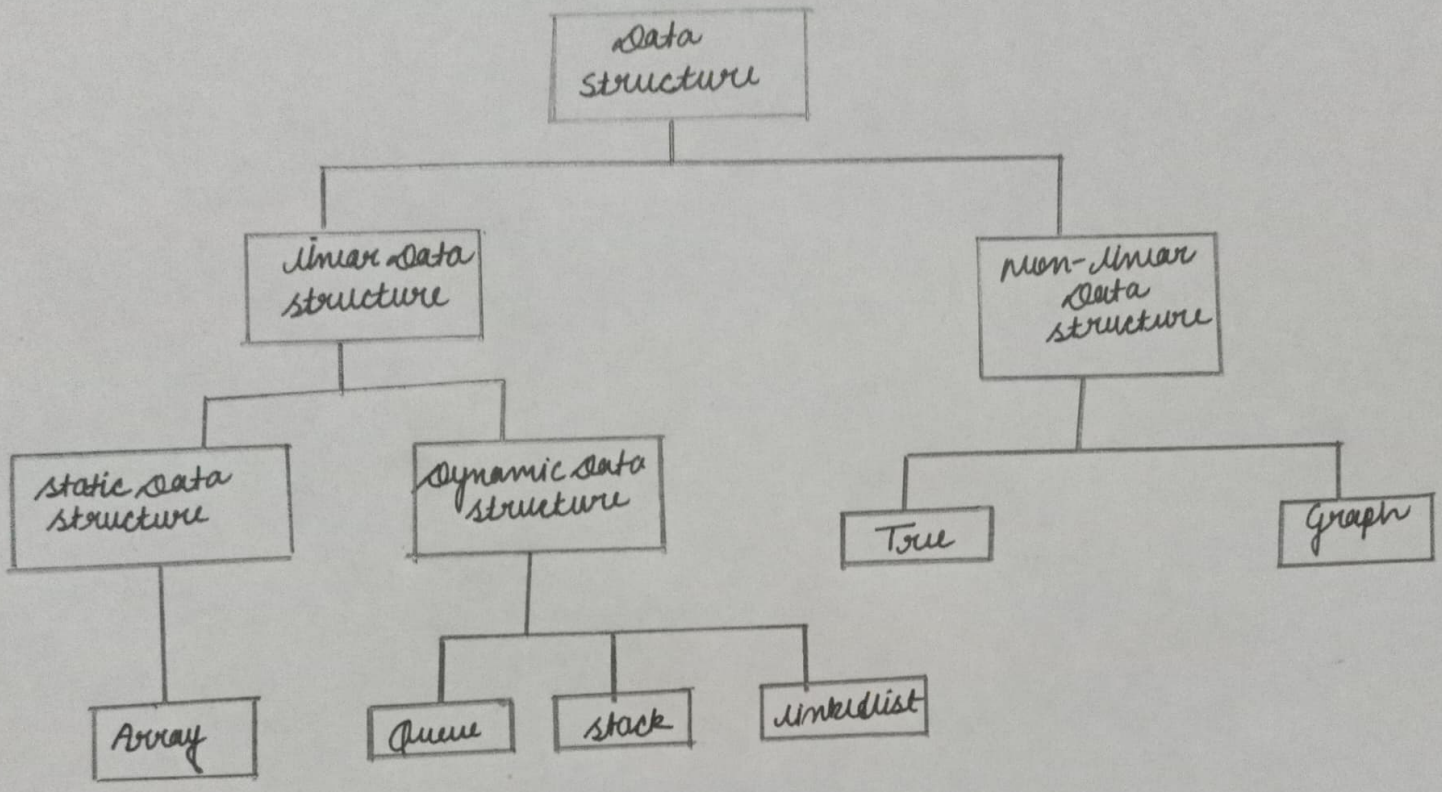
DSA Assignment

Name = Sanjyoti Singh
Section = 'C'
Course = BTech CS
Roll no = 55
University Roll no = 2115000911

① Define Data structure. Discuss various classifications of Data structure with suitable examples? Also, explain various operations of Data structure.

A Data structure is a storage that is used to store and organize Data. It is a way of arranging Data on a computer so that it can be accessed and update efficiently.
It is also used for processing, retrieving and storing Data.

Classification of Data structure:



Operations of Data structure:

① Searching: we can easily search for any element in Data structure.
② Sorting: we can easily sort Data
③ Insertion: we can insert any type of Data.
④ Deletion: we can easily remove/delete any element from Data.
⑤ Traversing: It means to visit the elements stored, it visits Data in a manner.
⑥ Create: It reserves memory for program elements by declaring them.
⑦ update: It ~~update~~ updates the Data in Data structure.

② What do you mean by time complexity? Define worst case, best case and average case with examples Also, Discuss in detail all types of Asymptotic notations with diagrams?

Time complexity is the number of operations an algorithm performs to complete it task. The Algo. that performs the task in the smallest no. of operations is considered the most efficent one in terms of time complexity.

cases of time complexity

① Worst-case:

Big-O Notation: We define an Algo. worst & case time complexity by using Big-O Notation which determines the set of functions grows slower than or at the same rate as the expression. Furthermore, it explains the maximum amount of time an algorithm requires to consider all input values.

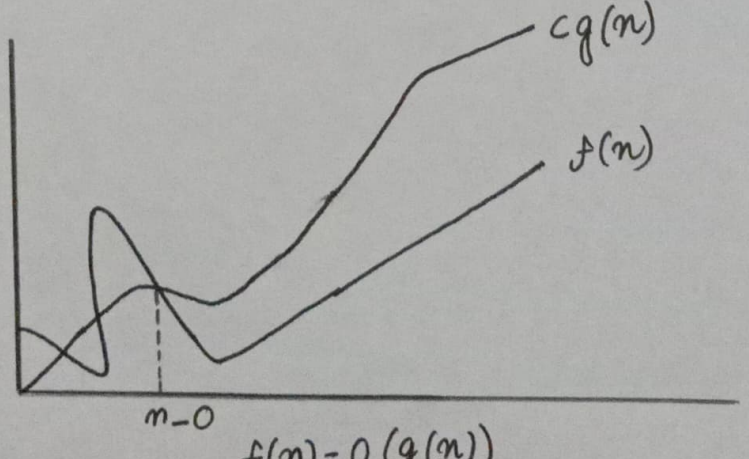
② Omega Notation (Ω)/Best case: The Omega Notation defines whether the set of functions will grow faster or at the same rate as the expression. it explains minimum amount of time an Algo. requires to consider all input values.

③ Theta Notation (Θ)/average case: It defines when the set of functions lies in both Big-O and Ω notation. then theta notation is used.

Diagrams of Asymptotic Notations:

① Big-O notation:

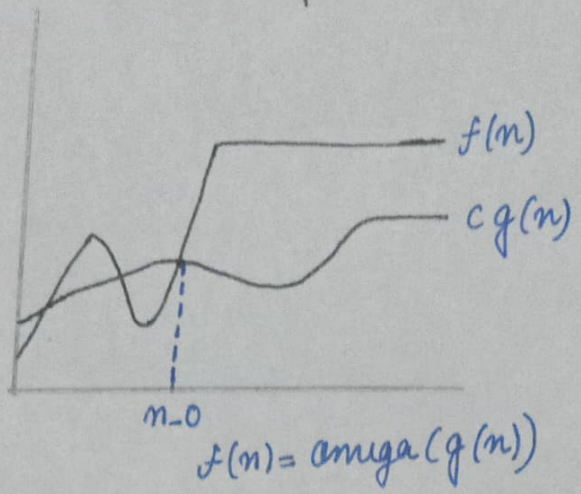
If $f(n)$ describes the running time of algo. $f(n)$ is $O(g(n))$ if there exist a +ve constant C and n0 such that $0 \leq f(n) \leq cg(n)$ for all $n \geq n0$.



$f(n) = O(g(n))$

Omega Notation (Ω-Notation):

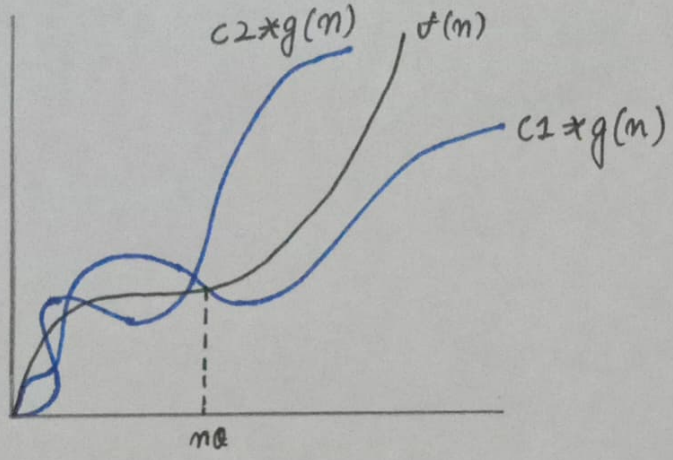
Let g and f be the function from the set of natural numbers to itself. The function f is said to be Ω(g) if there is constant c>0, and natural no. n0 such that $c * g(n) \leq f(n)$ for all $n \geq n0$.



f(n) = omega(g(n))

Theta notation (Θ)

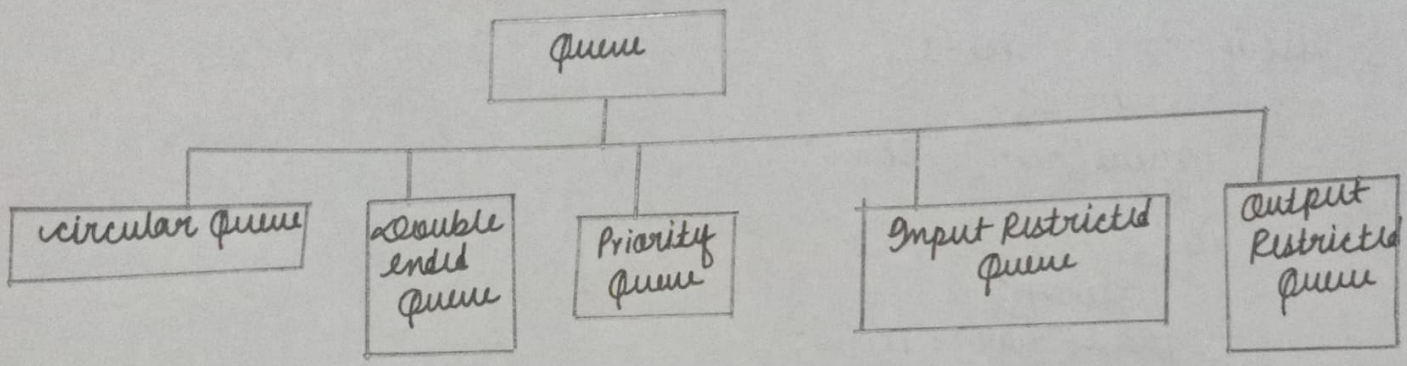
$c1 * g(n) \leq f(n) \leq c2 * g(n)$ for all $n \geq n0$



④ Define Queue? What are the various types of Queues? Also, write code to implement enqueue and dequeue methods in the circular queue using array.

A Queue is a linear structure that follows a particular order in which the operations are performed.
The order is FIFO (first in first out).

Types of Queues:



Program:

```
public class CircularQueue{
private int[] queue;
private int rear;
private int front;
private int size;
public CircularQueue(int n){
        size=n;
    queue = new list[size];
     rear= front=-1;
}
  private int delete(){
  if(front==-1 || rear==-1){
                System.out.println("Queue is Empty:(");
              }else if (rear==front){
                item = queue[front];
                 rear= front=-1;
              }else if (front=size-1){
                      item=queue[front];
                        front=0;
                    }else{
                        item = queue[front];
```

```
front++;
} return-1;
} public void insert (int item){
if(front==rear+1||size==rear+1){
        System.out.println("queue is Full:(");
    } else if(front==-1&&rear==-1){
            front=rear=0;
            queue[rear]=item;
} else if(rear==size-1){
        rear=0;
        queue[rear]=item;
    } else {
        rear++;
        queue[rear]=item;
    }
  }
  private void print(){
        if(front==-1)
          system.out.println("queue is Empty!");
          return;}
        if(rear>=front){
              for(int i=front;i<=rear;i++){
          System.out.println(queue[i]+" ");
        }
        } else {
            for(int=front;i<size;i++){
              System.out.print(queue[i]+" ");
            }
            for(int i=0;i<=rear;i++){
              System.out.print(queue[i]+" ");
            }
        }
          System.out.println();
        }
```

```
//Driver code
public static void main (String args[]){
  Circular Queue cqueue = new CircularQueue(5);
  cqueue.print();
  cqueue.insert(0);
  cqueue.print();
  cqueue.insert(10)
  cqueue.print();
  cqueue.insert(20);
  cqueue.print();
  cqueue.insert(30);
  cqueue.print();
  cqueue.delete();
  cqueue.print();
  cqueue.insert(40);
  cqueue.print();
 }
}
```

⑤ What is Recursion? Explain the advantages and disadvantages of using recursive formulation compared to the non-recursive formulation. Also, give suitable algorithms for linear recursion and binary recursion.

① When a function call itself again and again is called Recursion.
② It is a substitute of iteration.
③ All iterative programs are not recursive, but recursive programs are iterative.
④ We can use Recursion only in the logic in which previous calculation plays important role in next calculation.

Example:

```
int fact (int n)
{ if (n==0)
    return 1;
  else
    return n * fact (n - 1);
  }
```

Advantage	Disadvantage
① The code may be easier to write.	① Recursive functions are generally slower than non-recursive.
② Reduce unnecessary calling of function.	② It may require a lot of memory space to ~~first~~ hold intermidate results on system stacks.
③ Very useful in solving DSA problems	③ Hard to analyze and understand problems.

① Linear Recursion:

Algorithm: Reverse Array (A, i, j);

Input: an array A and non negative integer indices i and j.

Output:

The reversal of the element in A starting at index i and ending at index j.

if $i < j$ then

Swap A[i] and A[j]

ReverseArray (A, i+1, j-1)

return

End of if statement

Finished

EXIT.

② Binary Recursion:

Algorithm: BinaryFib(k)

Input: non negative integer k

Output:

The Kth Fibonacci number F_k.

If k=1

Then return k

Else

return BinaryFib(k-1) + BinaryFib(k-2)

end of if statement

[finished]

EXIT

Differentiate between stack and linked list? Write the program to implement the stack using an array.

Stack	Linked List
① An abstract data type that serves as a collection of elements with two principal operations which are push and pop.	① A linear collection of data elements whose order is not given by their location in memory.
② Push pop and peek are the main operations performed on stack.	② Insert, delete and traversing are the main operations performed on a linked list.
③ can read the topmost element.	③ It is required to traverse through each element to access a specific element.

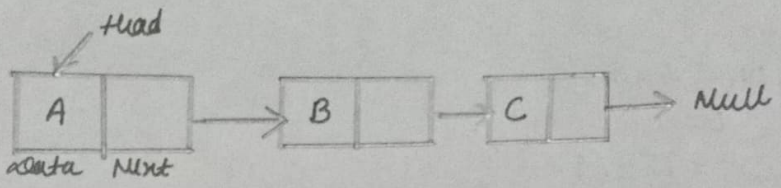
Program:

```
class stack1
{ private int s[];
  private int top;
  private int maxsize;
  //creating a stack
 Stack1(int size)
 { s=new int[size];
   maxsize=size;
   top=-1;
  } //push method
 public void push(int item)
       if(top==maxsize-1)
       System.out.println("Stack overflow");
    //terminates the program
       System.exit(1);
    } //inserting element on top of stack
    System.out.println("inserting"+item);
    s[++top]=item;
    } //pop method
```

```
public int pop()
{ if (top==-1)
 { System.out.println("stack underflow");
 // terminates the program
    System.exit(0);
  } // pop
    returns [top--];
   }
 //display elements of stack
 public void traversing()
{ for(int i= top; i>=0;i--)
{ System.out.println(s[i]+", ");
}
} public static void main (string args[]){
          Stack1 st= new Stack1(5);
           st.push(1);
           st.push(2);
           st.push(3);
           st.push(40);
           st.push(50);
           st.push(5);
         System.out.print("stack:");
         st.traversing();
       // remove element from stack
     st.pop();
     System.out.println("\n After popping out");
     st.traversing();
     }
     }
```

⑦ Define linked list? What are various types of linked list? write the method codes to implement insert at beginning, last and delete from location in single linked list?

A linked list is a linear data structre, in which the elements are not stored at contagious memory location.
The elements in linked list are linked using pointers as shown in below image:



Types of linked list:

① singly linked list
② Doubly linked list
③ circular linked list
④ Doubly Circular linked list

Algorithm INSERT_BEG(INFO, LINK, AVAIL, NEW, FIRST)

① [IS AVAIL Empty:]
IF AVAIL = NULL then
write "overflow and EXIT
[End of if statement]

② [creation of new node]
Set New := AVAIL
Set AVAIL := LINK [AVAIL]
Set LINK [NEW] := NULL
Set INFO [NEW] := ITEM

③ [IS LIST EMPTY?]
IF FIRST = NULL then
Set FIRST := NEW
Else
Set LINK [NEW] := FIRST
Set FIRST := NEW
[End of if statement]

④ [Finished]
EXIT

Algorithm INSERT_LAST(INFO, LINK, NEW, FIRST, AVAIL, ITEM, PREV)

① [IS AVAIL Empty:]
IF AVAIL = NULL then
write "Overflow" and EXIT
[End of if statement]
② [creation of new node]
Set NEW := AVAIL
Set AVAIL := LINK[AVAIL]
Set LINK[NEW] := NULL
Set INFO[NEW] := ITEM
③ [IS LIST Empty?]
IF FIRST = NULL then
Set FIRST := NEW
Else
Set PTR := FIRST
Repeat steps while PTR != NULL
Set PREV := PTR
Set PTR := LINK[PTR]
[End of while loop]
④ set LINK[PREV] := NEW
⑤ [Finished]
EXIT

Algorithm DELETE_LOC (INFO, AVAIL, LINK, LOC, TEMP)

① [IS LIST Empty?]
IF FIRST = NULL then
write "underflow" and EXIT
[End of if statement]
② If LOC = NULL then
write "invalid position and EXIT"
[End of if statement]
③ IF LINK[FIRST] = NULL then
Set LINK[FIRST] = AVAIL
Set AVAIL = FIRST
Set FIRST = NULL
~~Else~~ Else
Set LINK[PREV] := LINK[LOC]
Set LINK[LOC] = AVAIL
Set AVAIL = LOC

④ [Finished]
Exit.